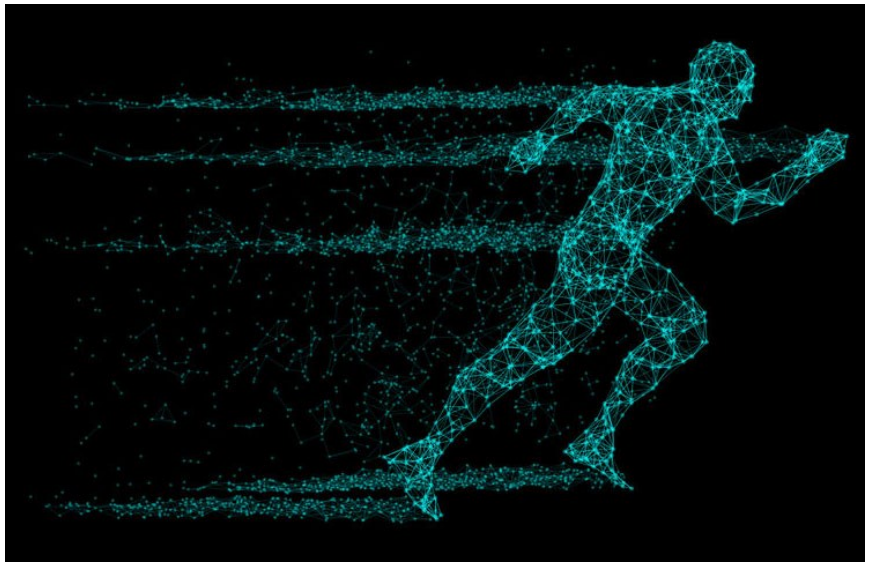
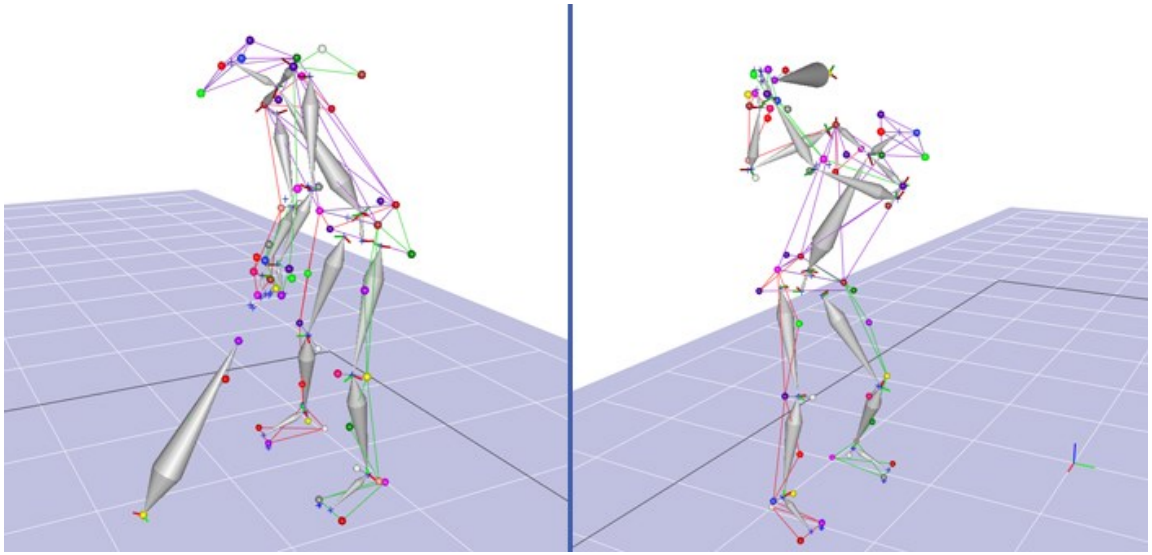
Chapter 6 – Motion Analysis

Comunicação Visual Interactiva





2D Motion

- **Analysis of video motion is important** – it allows to provide useful *features*, e.g., objects velocity, shape of the objects, and its trajectories
  - When observing the intensity (or color) of the pixels, it is possible to detect the presence of objects, and perform its recognition
- **Event detection** (e.g, scene transitions) – it allows to obtain several *clips* from long video sequences, being easier to perform operations, such as access, analysis and editing.
- **Four situations in which motion occurs** (pixel changes):
  - Fixed camera, only one object in motion, constant background
    - Easiest situation in video surveillance
  - Fixed camera, several objects in motion, constant background
    - Realistic situation in video surveillance
  - Motion cameras, scene approximately unchanged
    - This permits to obtain a large number of observations in the scene, e.g., building mosaics, computing depth of the objects, event detection such as *pan* ou *zoom* of the camera
  - Motion cameras, with several motion objects
    - Robotic vehicle moving in a traffic-intensive environment

Image Subtraction Methods

- Subtraction of consecutive images

$$\left|I_n(r,c) - I_{n-1}(r,c)\right| > T_n(r,c)$$

- Background subtraction

$$\left|I_n(r,c) - B_n(r,c)\right| > T_n(r,c)$$

$$B_{n+1}(r,c) = \begin{cases} B_n(r,c) & if \quad (r,c) \text{ it is an active pixel} \\ \alpha B_n(r,c) + (1-\alpha) I_n(r,c) & \text{otherwise} \end{cases}$$

Another algorithm for background subtraction:

$$(|I_n(r,c) < \min(r,c)| \vee |I_n(r,c) > \max(r,c)|) \wedge |I_n(r,c) - I_{n-1}(r,c)| > D(r,c)|$$

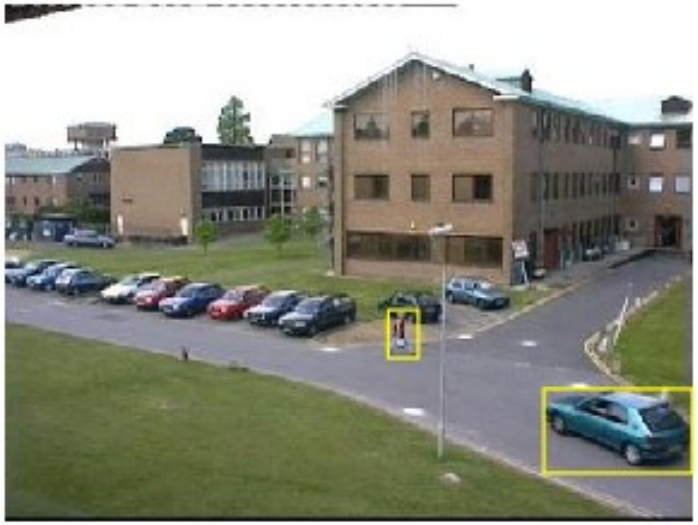
W4 – Who ? When ? Where ? What ?
(Real-time surveillance of people an their activities)

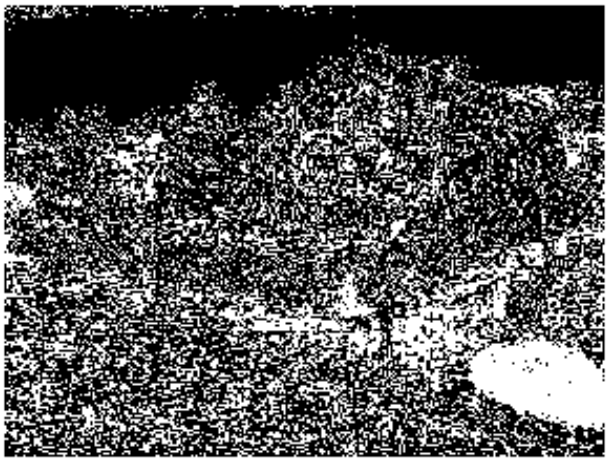
Please take a look to the following papers !

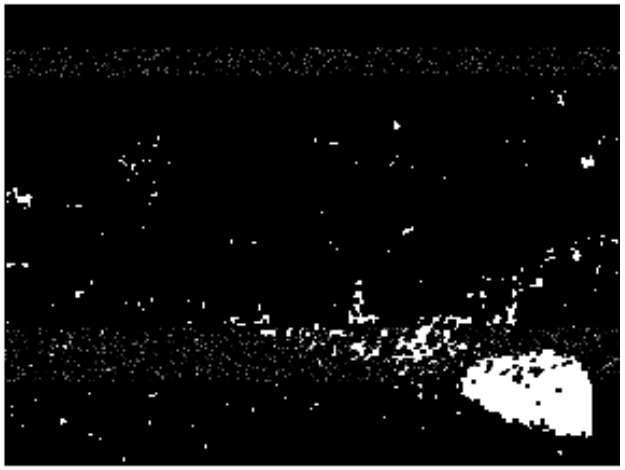
[1] J. C. Nascimento and J. S. Marques, "Performance Evaluation of Object Detection Algorithms for Video Surveillance", IEEE Transactions on Multimedia, vol. 8, no. 4, August 2006.

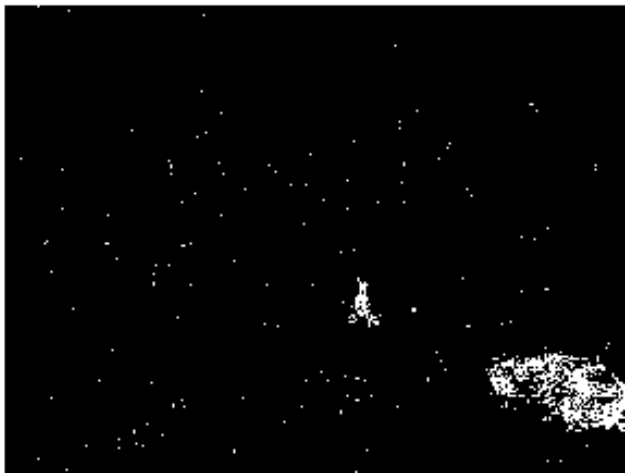
[2] I. Haritaoglu, D. Harwood and L. S. Davis, "W4, real-time surveillance of people and their activities", IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, August 2000.

Image Subtraction Methods

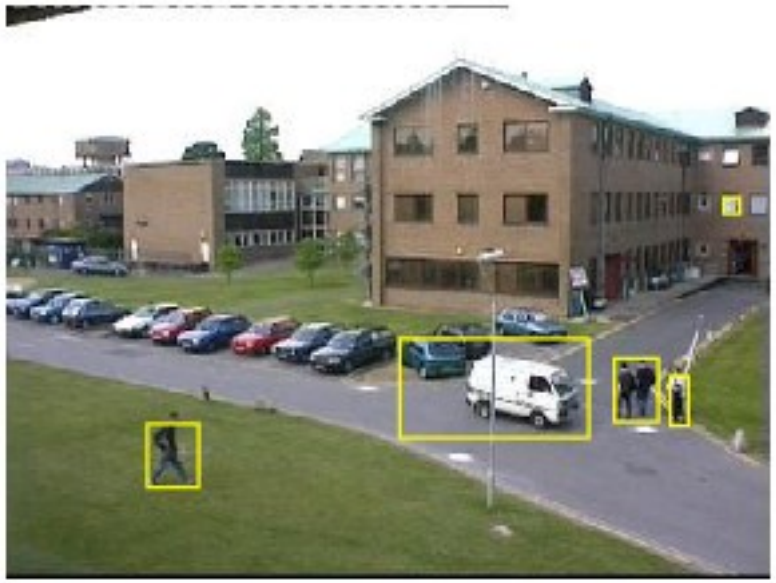


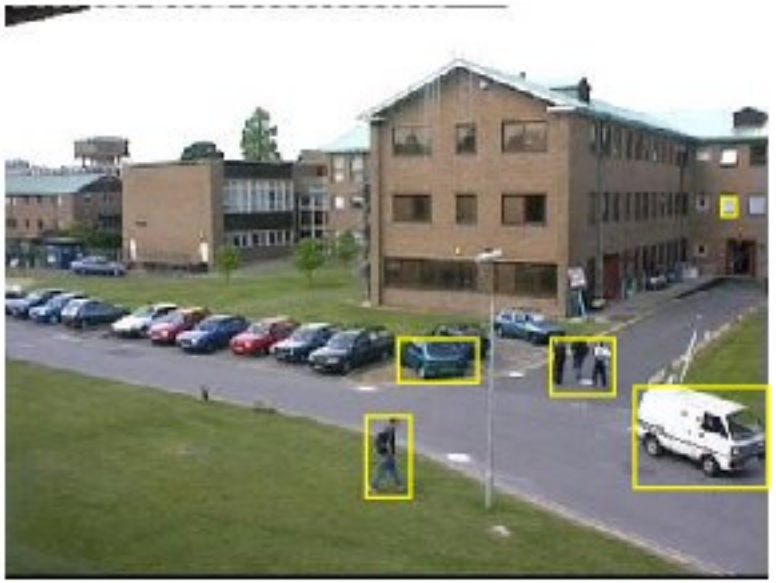




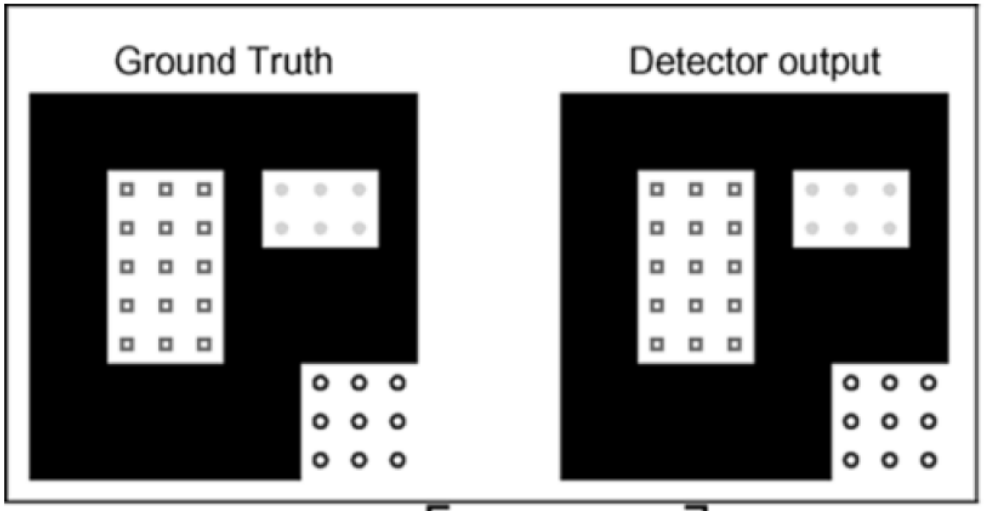


Bounding boxes or foreground regions detected in a sequence of images



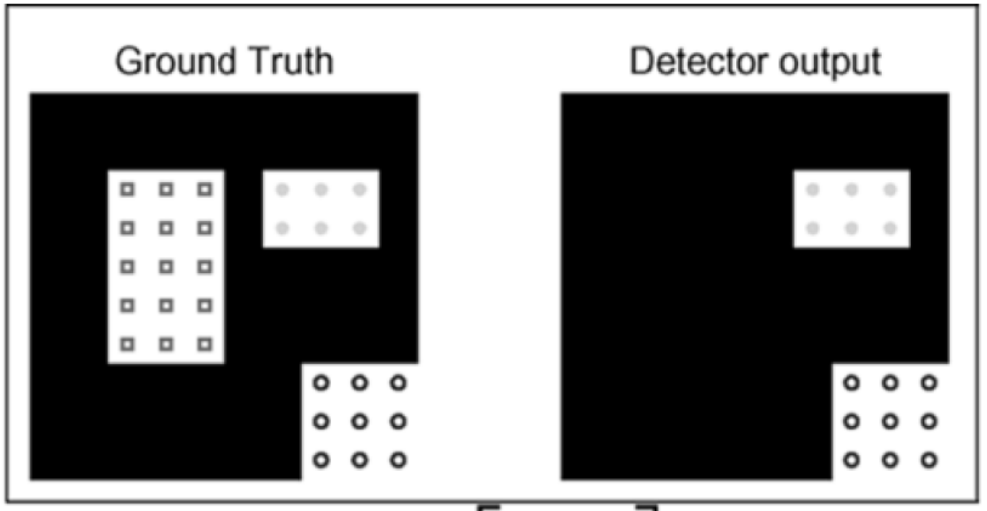


Association Matrix for event detection (1)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

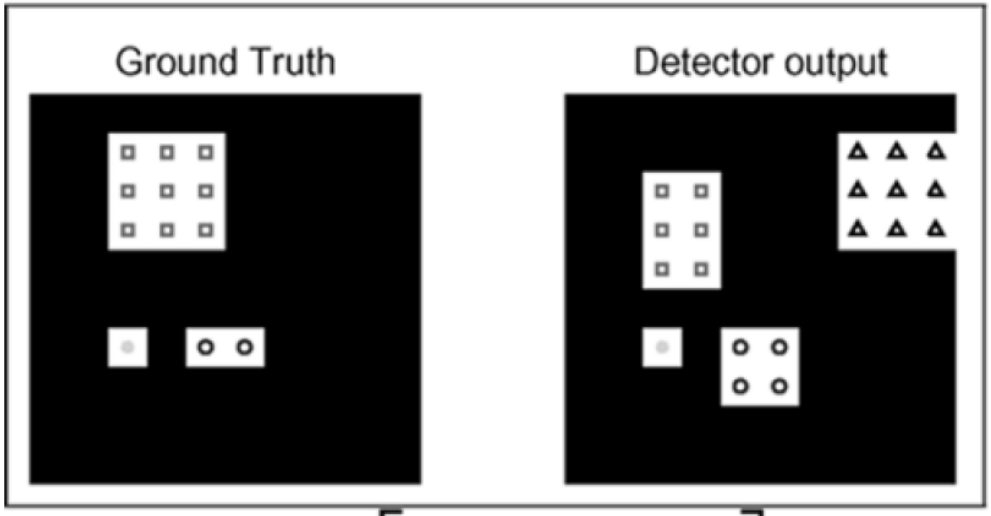
Three correct detections



$$\mathcal{C} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

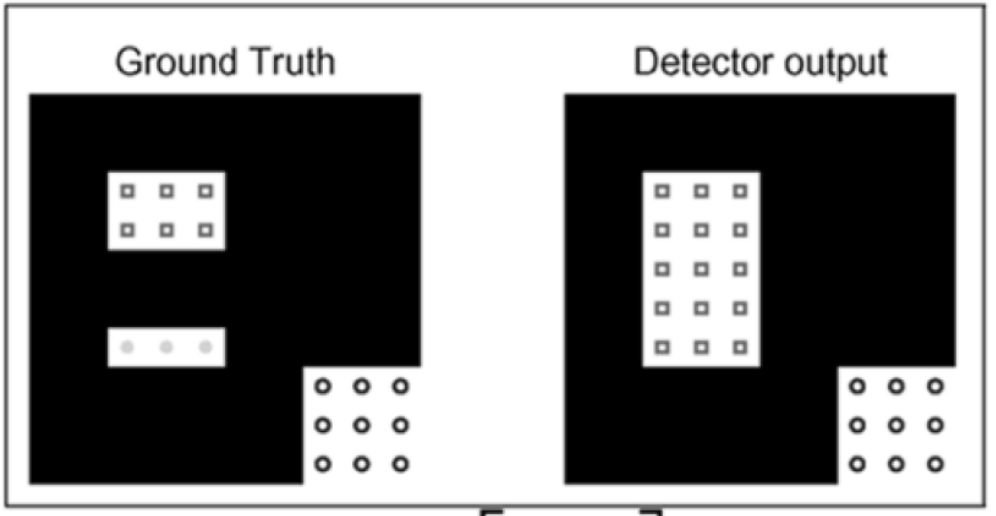
Two correct detections and
one mis-detection
(a line with zeros)

Association Matrix for event detection (2)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

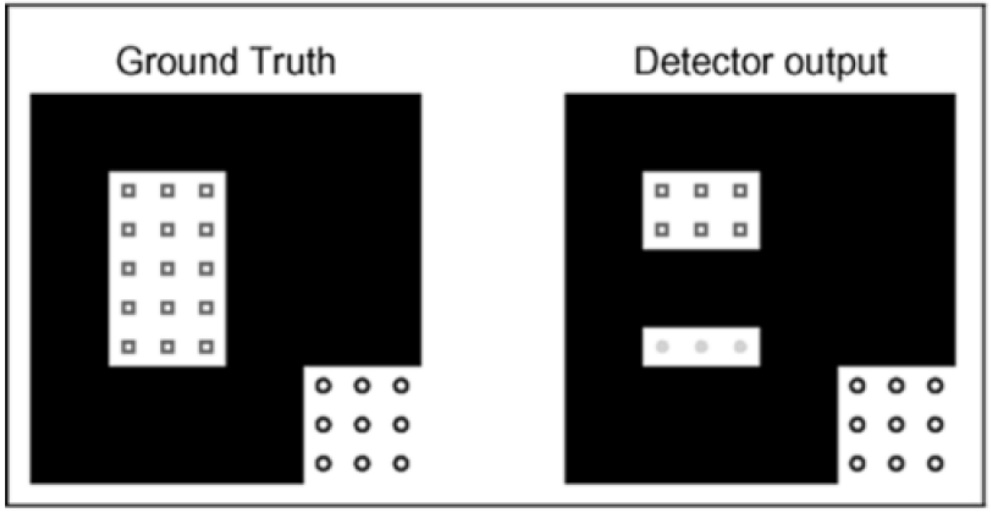
Three correct detections and
one false positive (a column with zeros)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

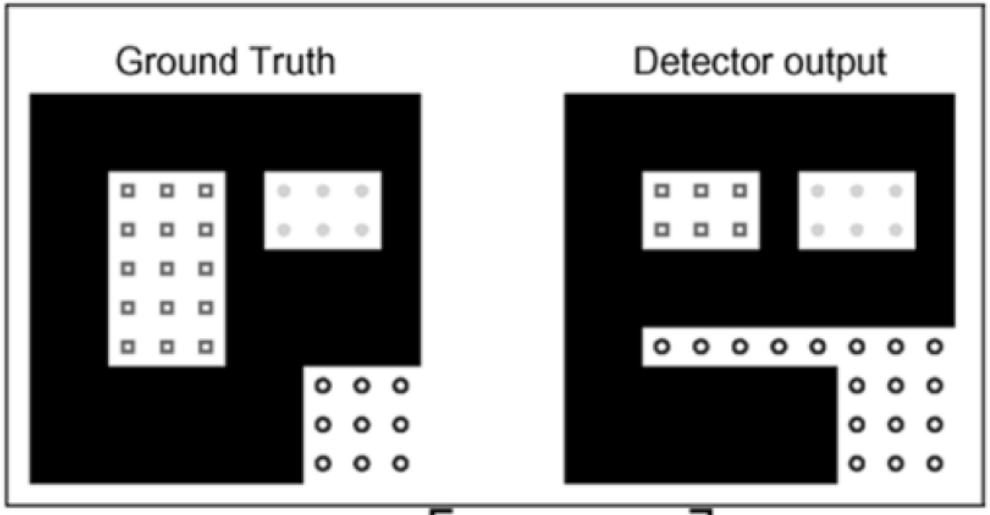
One correct detection and
one merge
(the sum of a column > 1)

Association Matrix for event detection (3)



$$\mathcal{C} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

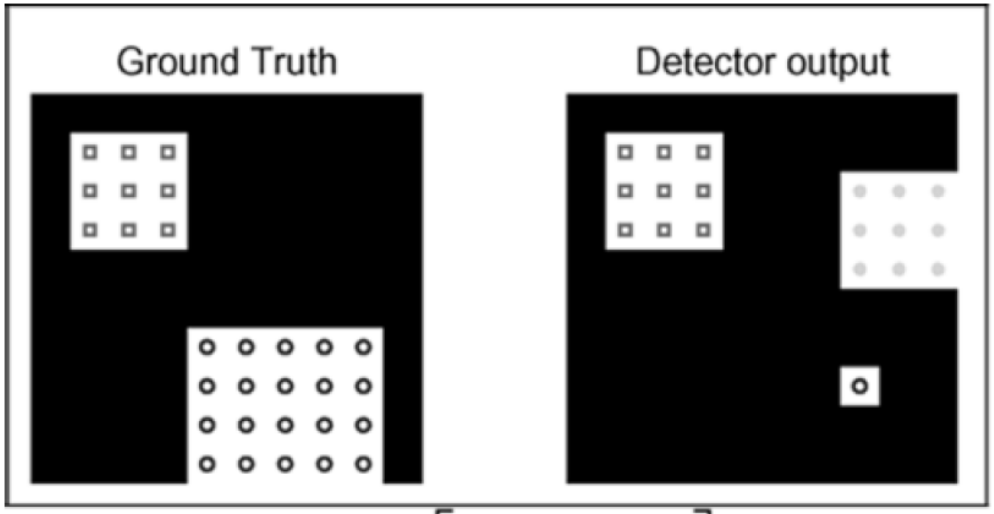
One correct detection and
one split
(the sum of a line >1)



$$\mathcal{C} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

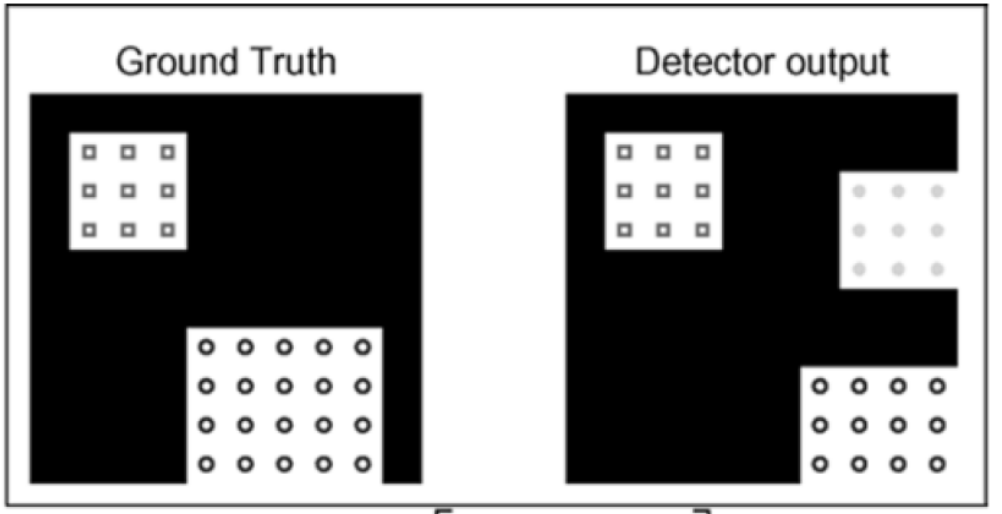
One correct detection and
one split-merge
(the sum of a line and a column > 1)

Association Matrix for event detection (with a threshold requirement)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

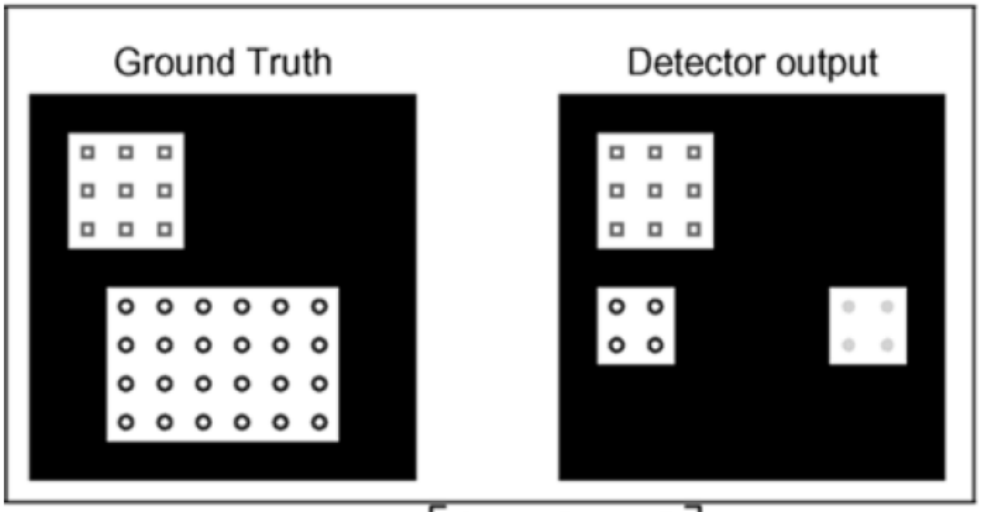
One correct detection and
one mis-detection (line with zeros)
and two false positive (colum with zeros)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

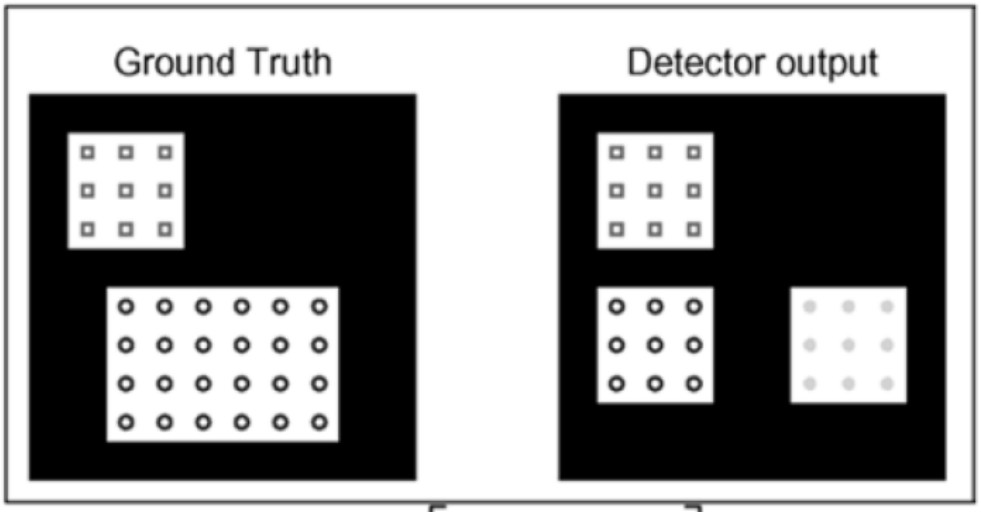
Two correct detections and
one mis-detection and one false positive
(column with zeros)

Association Matrix for event detection (with a threshold requirement)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

One correct detection and
one mis-detection (line with zeros)
and one false positive (column with zeros)



$$\mathcal{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

One correct detections and
one split (sum of a line > 1)

- **Input**: two monochromatic images $I_n(r,c)$ and $I_{n-k}(r,c)$ and threshold τ
- **Output**: binary image, I_{out} and a set of bounding boxes containing the localization of the detected objects

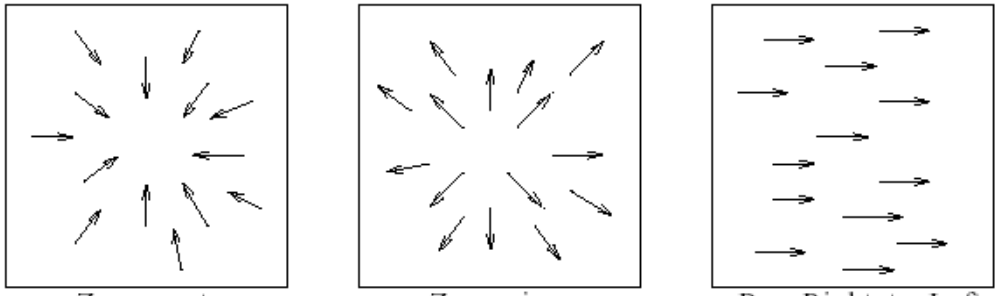
- Algorithm with 5 steps:

  1. Compute binary images (i.e., active pixels)

$$I_{out}(r,c) = \begin{cases} 1 & \text{if } \left|I_n(r,c) - I_{n-k}(r,c)\right| > \tau \\ 0 & \text{otherwise} \end{cases}$$

  2. Perform closed morphological operations using a structuring element
  3. Perform connected component analysis in I_{out}
  4. Remove regions with small area (noise)
  5. For each region, compute the corresponding *bounding box*

- A set of 2D vectors that represent the motion in the image (that may contain 3D objects) is known as **motion field.**

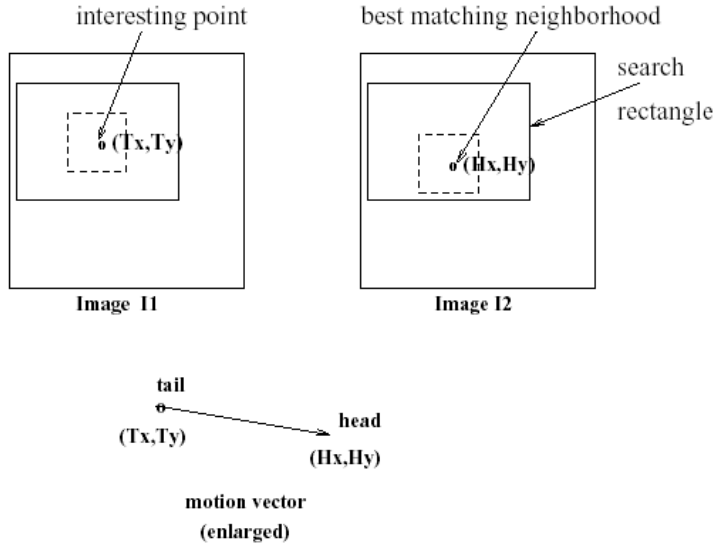


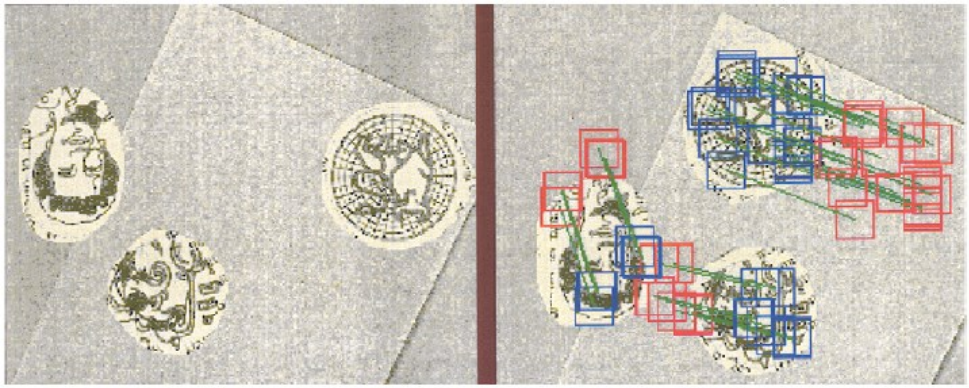
- The focus of expansion (**FOE**) is the point in the image where the vectors diverge. The focus of contraction (**FOC**) is the point in the image, in which the vetors converge.

- **Optical Flow** is the computed motion field, assuming that neighbor intensidades of the correspondent points remains unchangeable (i.e. the intensity is constant).

Sparse method – point matching

- Detect interest points in the 1st image (e.g., corner detection or regions centroids)
- For each point of interest detected in the 1st image (Tx, Ty) search in the 2nd image for the corresponding point (e.g., using matching)
- If the *match* is good, the point coordinates can define the begin of the motion vector (Hx,Hy)





Computation of interest points (example)

Find interesting points of a given input image.

```
procedure detect_corner_points(I, V);
{
      "I[r, c] is an input image of MaxRow rows and MaxCol columns"
      "V is an output set of interesting points from I."
      "τ is a threshold on the interest operator output"
      "w is the halfwidth of the neighborhood for the interest operator"
      for r := 0 to MaxRow - 1
        for c := 0 to MaxCol - 1
          {
          if I[r, c] is a border pixel then break;
          else if ( interest_operator (I, r, c, w ) ≥ τ1) then add [(r, c), (r, c)] to set V;
      "The second (r, c) is a place holder in case vector tip found later."
          }
}
real procedure interest_operator (I, r, c, w )
{
      "w is the halfwidth of operator window"
      "See alternate texture-based interest operator in the exercises."
      v1 := variance of intensity of horizontal pixels I1[r, c − w] ... I1[r, c + w];
      v2 := variance of intensity of vertical pixels I1[r − w, c] ... I1[r + w, c];
      v3 := variance of intensity of diagonal pixels I1[r − w, c − w] ... I1[r + w, c + w];
      v4 := variance of intensity of diagonal pixels I1[r − w, c + w] ... I1[r + w, c − w];
      return minimum {v1, v2, v3, v4};
}
```

Algorithm 17: Algorithm for detecting interesting image points.

Analogous algorithm – MPEG compression

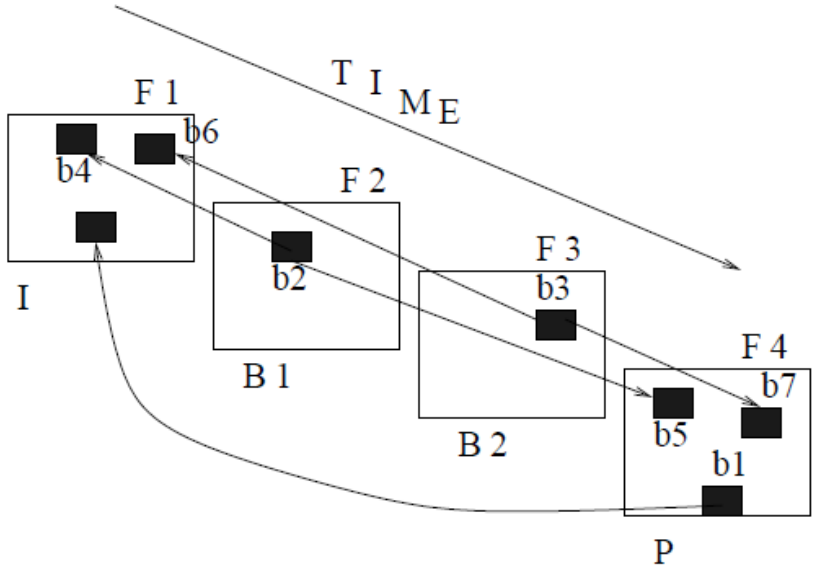


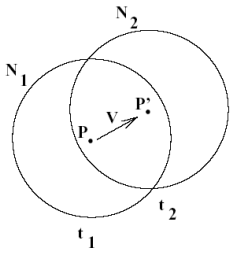
Figure 9.8: **A coarse view of the MPEG use of motion vectors to compress the video sequence of four frames F1, F2, F3, F4.** F1 is coded as an *independent* (I) frame using the JPEG scheme for single still images. F4 is a P frame *predicted* from F1 using motion vectors together with block differences: 16 x 16 pixel blocks (**b1**) are located in frame F1 using a motion vector and a block of differences to be added. *Between frames* B1 and B2 are determined entirely by interpolation using motion vectors: 16 x 16 blocks (**b2**) are reconstructed as an average of blocks (**b4**) in frame F1 and (**b5**) in frame F4. Between frames F2 and F3 can only be decoded after predicted frame F4 has been decoded even though these images were originally created before F4. Between frames yield the most compression since each 16 x 16 pixel block is represented by only two motion vectors. I frames yield the least compression.

- Image representation (continuous model) using Taylor series (1ª order)

$$f(x+\Delta x, y+\Delta y, t+\Delta t) = f(x,y,t) + \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y + \frac{\partial f}{\partial t}\Delta t + h.o.t.$$

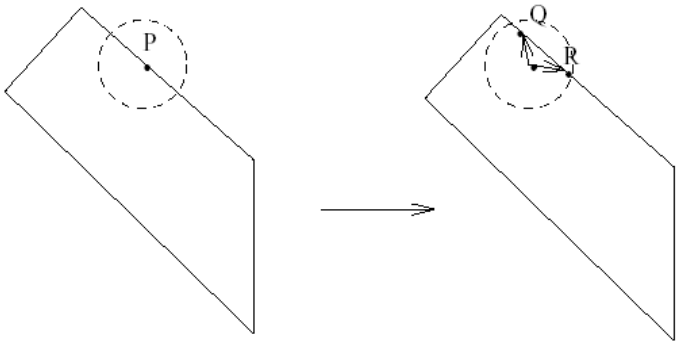
- Constant intensity hipothesis

$$f(x+\Delta x, y+\Delta y, t+\Delta t) = f(x,y,t) \longrightarrow$$

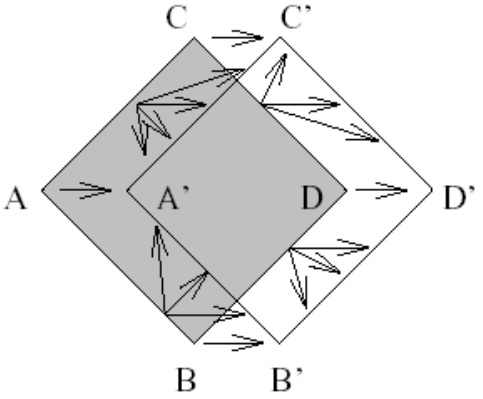


- Output:

$$-\frac{\partial f}{\partial t}\Delta t = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right) \circ (\Delta x, \Delta y)$$



Aperture problem



Importance of the corners

Fast Algorithm (Freeman *et. al.*)

- Compute the difference between the current and the previous image

$$d_n(r,c) = I_n(r,c) - I_{n-1}(r,c)$$

- For every pixels where the difference is not null (or large), enumerate every possible motion consistent with the observations:

1. If $d_n(r,c) < 0$ then the motion has the direction of the neighbor with larger intensity
2. If $d_n(r,c) > 0$ then the motion has the direction to the neighbor with smaller intensity

- In the previous step, consider four possible directions (four neighbor types): horizontal, vertical and two diagonals
- Consider the above 4 estimates as vector, and perform the sum (of the vectors) for each pixel
- Finally, estimate the optical flow for each pixel, computing the mean in the neighborhood of 3x3 pixels

Fast Algorithm (illustration)

Image I(t)

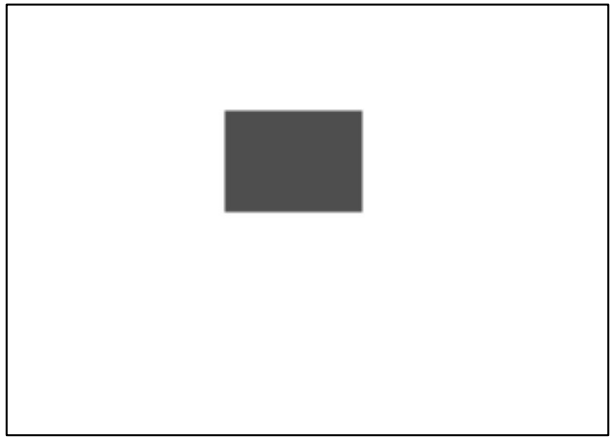
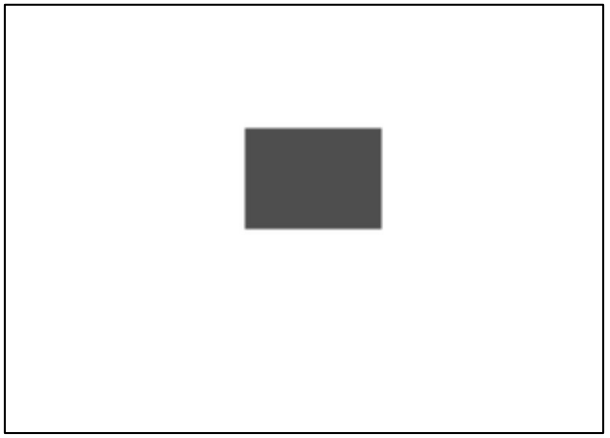
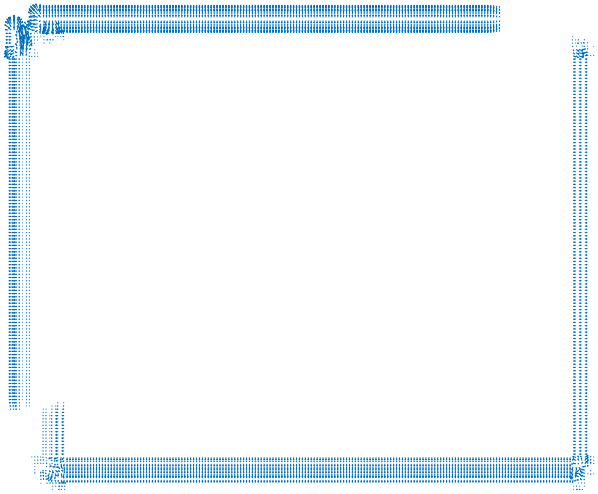


Image I(t+1)

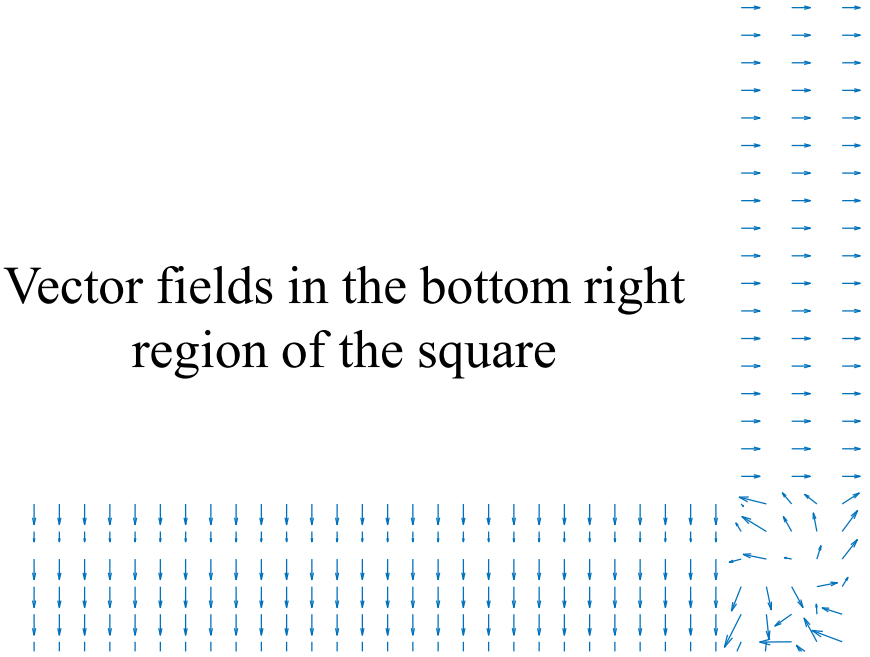


I(t+1) – I(t)

Vector fields in the square boundary



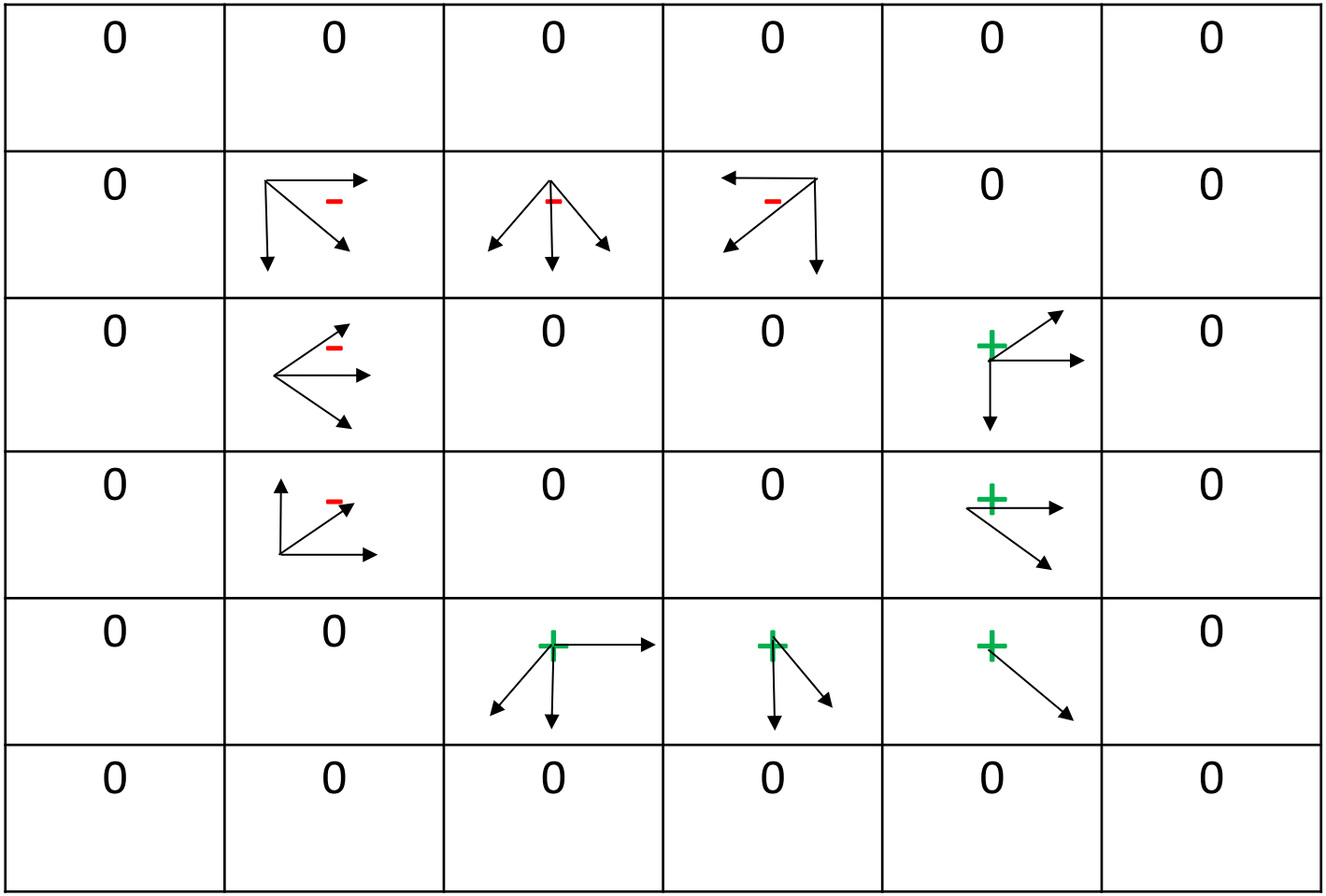
Vector fields in the bottom right region of the square



Fast Algorithm (illustration)

0	0	0	0	0	0
0	-	-	-	0	0
0	-	0	0	+	0
0	-	0	0	+	0
0	0	+	+	+	0
0	0	0	0	0	0

Fast Algorithm (illustration)



(considering the contribution of three neighborhood pixels)

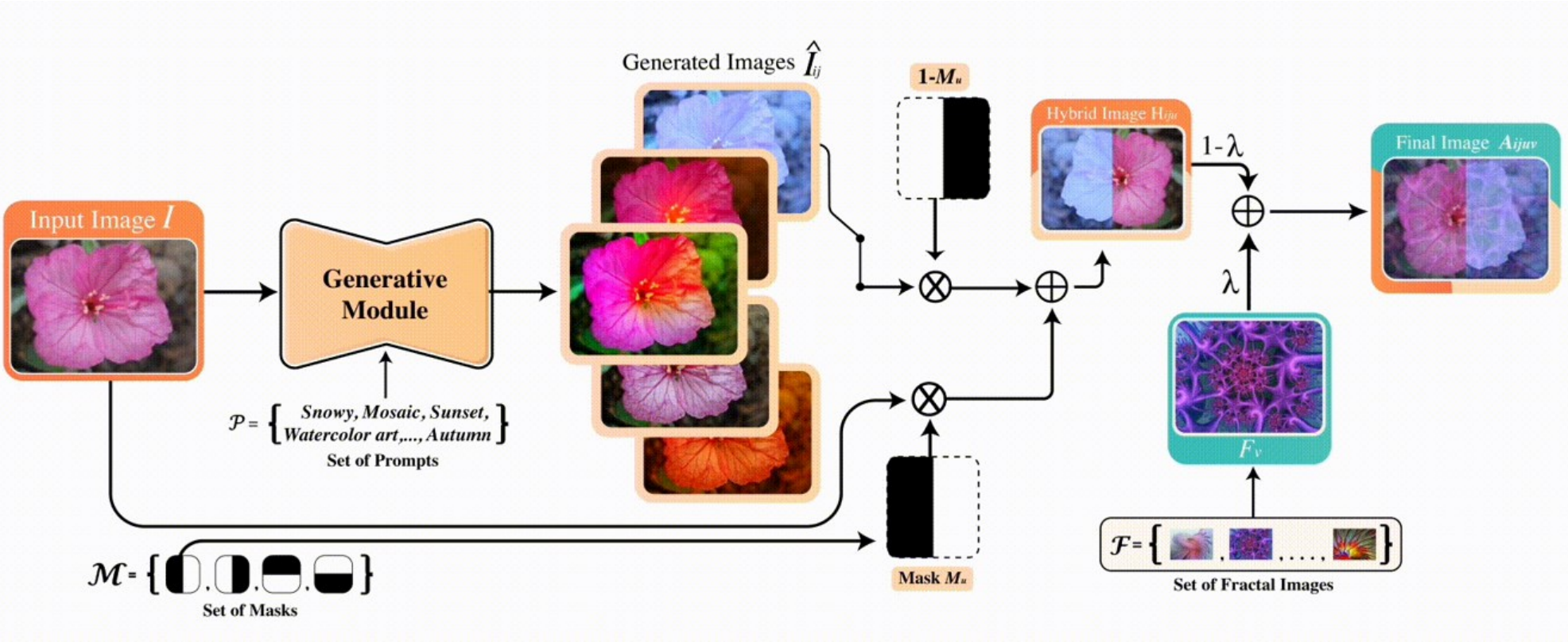
Freeman's Algorithm

W. T. Freeman, D. Anderson, P. Beardsley et al,
"Computer vision for Interactive Computer Graphics"

- Goal: Detection of large changes in the video
  - Scene change
  - Shot change
  - Pan
  - Zoom in e zoom out
  - Special effects
    - fade in, fade out $C = A\left(1-\frac{t}{T}\right)u(T-t) + B\frac{t-T}{T}u(t-T)$
    - Dissolve $C = A\left(1-\frac{t}{T}\right) + B\frac{t}{T}$
    - wipe

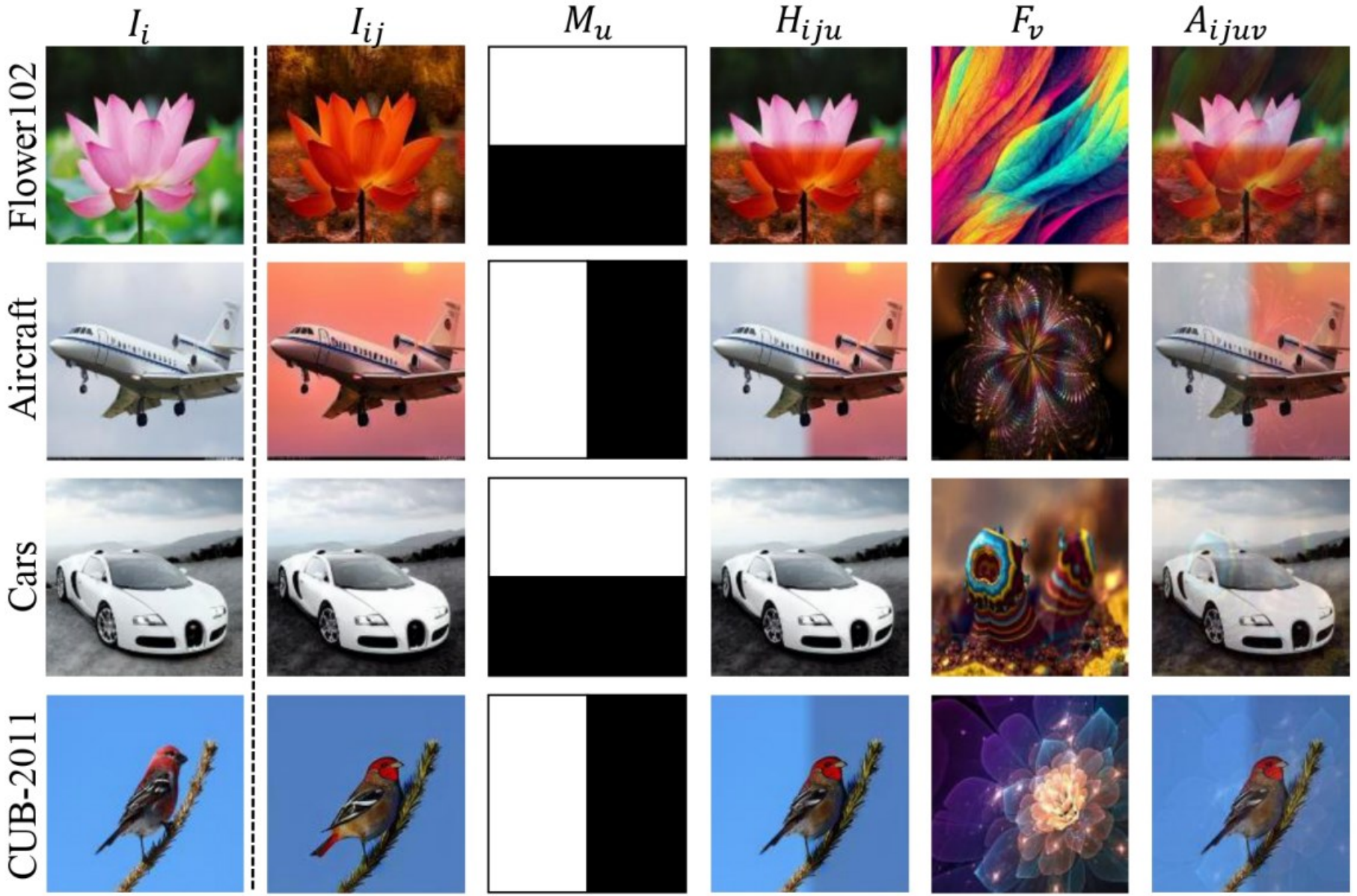
Special effects

DiffuseMix (i)



Khawar Islam et al "DiffuseMix: Label-Preserving Data Augmentation with Diffusion Models", CVPR 2024.

DiffuseMix (ii)



Metrics for Segmenting Video Sequences

- Simple solution (not convenient)

$$d(I_1, I_2) = \frac{\sum_{r=0}^{N-1}\sum_{c=0}^{M-1}\left|I_1(r,c) - I_2(r,c)\right|}{NM}$$

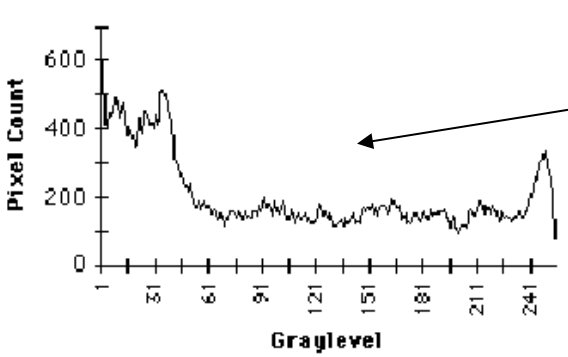
- Better solution: partition of the image in rectangular boxes and compute the mean and covariance for each block

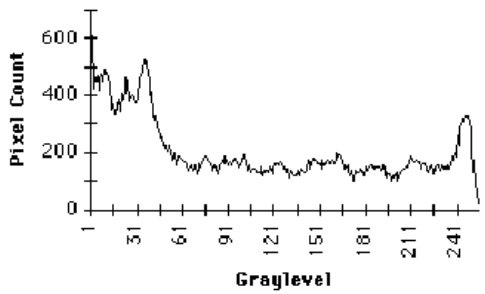
$$d_{block}\left(B_i, B_j\right) = \begin{cases} 1 & if \quad r > \tau \\ 0 & if \quad r \le \tau \end{cases} \qquad r = \frac{\left[\frac{\sigma_i^2 + \sigma_j^2}{2} + \left(\frac{\mu_i - \mu_j}{2}\right)^2\right]^2}{\sigma_i^2 \sigma_j^2}$$

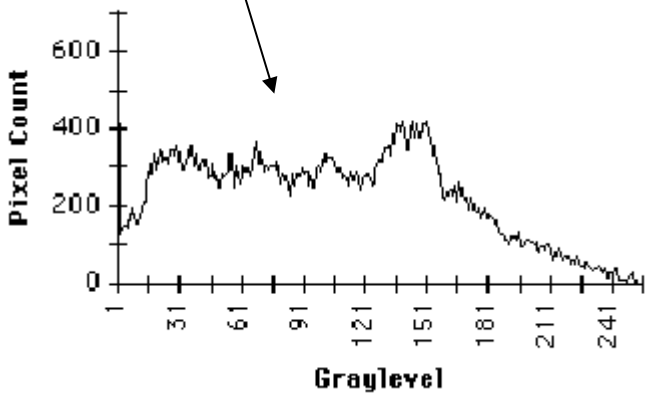
$$d(I_1, I_2) = \sum_{B_{1i} \in I_1; B_{2i} \in I_2} d_{block}(B_{1i}, B_{2i})$$

Other method – Histogram distance

Histogram Distances (other method)



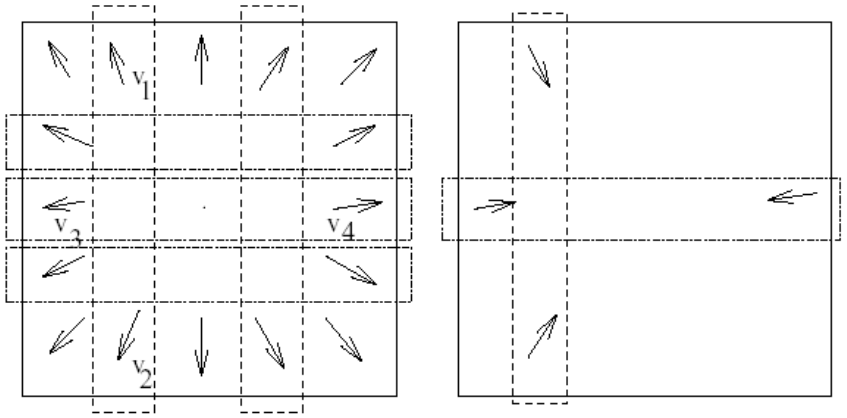




How to detect (certain) camera effects

- Zoom detection

The effects of the "zoom in" and "zoom out" can be detected whenever the sum of the vector fields are (approximately) zero (see left and right figures)



horizontal/vertical heuristic

$$|v_{1r} - v_{2r}| > \max\{|v_{1r}|, |v_{2r}|\}$$

$$|v_{3c} - v_{4c}| > \max\{|v_{3c}|, |v_{4c}|\}$$

- Shot detection
- Keyframes
- Video Clips; random access
- Image data base

The MPEG7 contains a set of descriptors that allow to represent audio-visual files (the videos are a particular case) in a standard way, and thus to facilitate the information search based on the contents (instead of using keywords).

Computer Vision for Interactive Computer Graphics

- Human Computer Interface – HCI
- The computers are able to interpret motion, gestures and gaze (of the users)
- Visual algorithms are based on:
  - Objects tracking;
  - Pattern (or shape) recognition;
  - Motion analysis.
- In interactive graphics applications, these algorithms should be: robust, fast and run in usual hardware.
- Fortunately, in most interactive applications the computer vision problem is not so difficult :
  - Constrain the number of possible interpretations
  - Provides useful visual feedback

Applications (I)

- Interactive applications are challenging, because:
  - Response time should be very fast
  - The computer vision algorithms should be robust for different types of persons and different backgrounds.
  - Must be cheap, e.g., a joystick or a TV command. Even with a larger number of functionalities, the consumer does not want to pay more.

- Advantages:

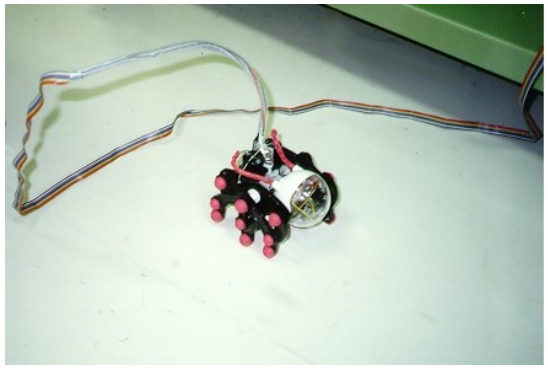
  - The context of the application usually constrains the possible visual interpretations (e.g., in a game context we know that a person is running; in this case, we only need to define the velocity)

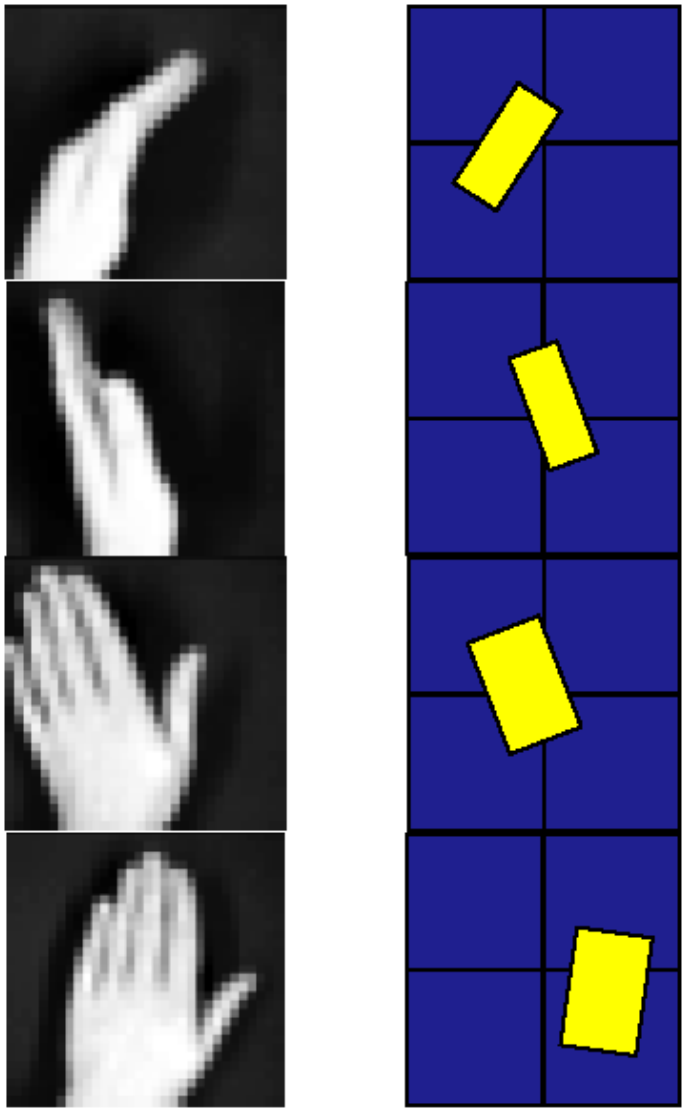
  - There is a human in the loop. The user can explore the rapid visual feedback provided to him via graphic display to change his gesture, to achieve his goals (e.g., if the player bends to make a turn and notices that he is not bowing enough, then he should bow more.)

Applications

- There is a niche applications for fast computer vision algorithms that are not complex. These algorithms take advantage of of the simplicity of the graphical interfaces.

- Other application is *active vision,* with real-time response of real vision systems.





Applications

- Vision (e.g. camera) can be a computer interface device
  - Body position
  - Head orientation
  - Gaze
  - Gestures
- Applications may include
  - Games or machine control (via computer)
  - Computer (natural) interface
- Instead of using buttons, the player can mimic gestures or actions that the computer is able to recognize
- The user can provide machine comands using manual gestures (useful for surgical intervention, disabled persons etc).